

GOOGLE MAPS + GEOCODING - COMO LIGAR (PASSO A PASSO)

Estado em 12/09/2026: o app já fala com o servidor para as duas coisas que dependem do Google, e as duas funções **estão publicadas** no Supabase:

Função	O que faz	API do Google
travel-times	tempos de deslocamento entre paradas (matriz com trânsito)	Routes API (computeRouteMatrix, TRAFFIC_AWARE)
geocode	endereço para coordenada (o pino no mapa)	Geocoding API

Falta só a chave. Sem ela, as duas respondem 501 sem-chave-do-google e o app segue funcionando como antes (estimativa de linha reta, endereço sem pino) — nada quebra. Com a chave, a operação passa a ter trânsito real e pino no mapa.

0. Qual conta, e como não perder o link

Decisão de 12/09/2026: o projeto fica na conta da **Automa Digital** (automadigitalsup@gmail.com) — é o que destrava hoje. Quando o app for vendido para o cliente, ele cria um projeto próprio na conta dele e a gente **troca as duas chaves**: trocar chave não é migrar projeto, é colar dois textos novos (um no app.json para o build, outro no segredo do Supabase). Projeto do Google não se transfere entre contas comuns sem organização — recriar é mais rápido e mais limpo.

Se o link abrir na conta errada, ou "perder o link" ao trocar de conta: o Google descarta o endereço na troca e cai na tela inicial do console. Três saídas:

Saída	Como
Rápida	Janela anônima (Ctrl+Shift+N), login só na conta certa, e colar o link lá dentro
Definitiva	Perfil separado no Chrome: ícone do perfil -> Adicionar -> Continuar sem uma conta -> login só na conta certa (use sempre esse perfil para o console)
Pela URL	Acrescente ?authuser=automadigitalsup@gmail.com no fim do endereço (se já houver ?, use &authuser=...)

E, dentro do console, **busque a API pelo nome na Biblioteca** em vez de colar links — não existe risco de digitar errado.

Cartão e custo: o faturamento exige cartão, e conta nova do Google Cloud costuma vir com **US\$ 300 de crédito de teste**. Dentro da faixa gratuita o gasto é US\$ 0; o cartão só entra se estourar — por isso as duas travas abaixo.

Trava dura contra fatura (faça junto, leva dois minutos):

Routes API -> cota de requisições por dia -> **Editar cota** -> **1.000/dia**. Faça o mesmo para **Geocoding API** com **500/dia**. (O volume real hoje é de ~360 eventos de rota por dia e cerca de um endereço novo por dia — então o teto fica 3x acima do uso e ainda trava o estrago.)

avisos em 50%, 90% e 100%. Atenção: alerta **avisa, não bloqueia** — quem bloqueia é a cota.

- **Cota diária:** APIs e serviços -> **Cotas (Quotas & System Limits)** -> filtrar por
- **Alerta de orçamento:** Faturamento -> **Orçamentos e alertas** -> orçamento de **US\$ 5** com

1. Quanto custa (fonte: tabela oficial do Google, consultada em 12/09/2026)

Serviço	Grátis por mês	Depois
Maps SDK for iOS (o mapa dentro do app)	ilimitado	-
Routes API (matriz de rotas)	10.000 chamadas	US\$ 5,00 por 1.000
Geocoding	10.000 chamadas	US\$ 5,00 por 1.000

No volume do Pack & Paws — uma rota por dia com 12 paradas, mais o recálculo de chegada — dá cerca de **7.900 eventos por mês**, ou seja **US\$ 0**. Se a operação dobrar: US\$ 10 a 25/mês.

2. Criar o projeto e ligar a cobrança

Decida com o cliente: quem cria o projeto é quem fica com a conta e o cartão. Na proposta, o consumo do Google é cobrado **na conta dele**.

faturamento com cartão. Sem isso nenhuma chave funciona, nem dentro da faixa gratuita.

1. Entre em `console.cloud.google.com` com a conta Google escolhida.
2. Barra superior, seletor de projeto -> **Novo projeto** (*New project*).
3. Nome: Pack & Paws Club -> **Criar** (*Create*).
4. Com o projeto selecionado, abra **Faturamento** (*Billing*) e vincule uma conta de

3. Habilitar as três APIs

APIs e serviços (*APIs & Services*) -> **Biblioteca** (*Library*), buscando uma por uma:

1. **Maps SDK for iOS**
2. **Geocoding API**
3. **Routes API**

Em cada uma: **Ativar** (*Enable*).

4. CHAVE 1 - a do aplicativo (mapa dentro do celular)

APIs e serviços -> **Credenciais** (*Credentials*) -> + **Criar credenciais** -> **Chave de API** (*API key*). Copie a chave (AIza...) e clique em **Editar chave de API**:

+ **Adicionar** -> ID do pacote: `br.com.automadigital.app`

Maps SDK for iOS

- **Nome**: Pack & Paws - iOS Map
- **Restrições de aplicativo** (*Application restrictions*): **Apps para iOS** (*iOS apps*) ->
- **Restrições de API** (*API restrictions*): **Restringir chave** -> marque **somente**
- **Salvar** (*Save*)

Essa chave entra no build (ela é pública por natureza — vive dentro do binário; quem a protege é a restrição de bundle ID):

```
python3 /opt/data/pack-and-paws/build/set_google_config.py --maps-key AIza...
```

...e depois um build novo. Sem o build novo, o mapa dentro do app continua sendo o do iPhone (Apple Maps).

5. CHAVE 2 - a do servidor (trânsito + endereço)

Mesma tela, + **Criar credenciais** -> **Chave de API**:

então restrição por IP só quebra sem proteger

- **Nome:** Pack & Paws - Server (Routes + Geocoding)
- **Restrições de aplicativo:** **Nenhuma** (*None*) — o servidor do Supabase não tem IP fixo,
- **Restrições de API:** marque **Routes API** e **Geocoding API**
- **Salvar**

Trava de segurança (faça isso): em **APIs e serviços** -> **Cotas** (*Quotas*), defina um limite diário por API (ex.: 5.000 por dia). É a única proteção real contra fatura surpresa.

6. Colocar a chave no servidor

```
supabase secrets set GOOGLE_ROUTES_KEY=Aiza... GOOGLE_GEOCODING_KEY=Aiza... --project-ref
bhuexxjcrjdhkmsvagd
```

Pode ser a **mesma** chave nos dois nomes (a função geocode também aceita GOOGLE_ROUTES_KEY como reserva). Segredo entra na hora: **não precisa republicar** as funções.

7. Provar que ligou

```
cd /opt/data/pack-and-paws/mobile && node scripts/google-smoke.mjs
```

Esperado: duas linhas [OK] — uma com o tempo de trânsito entre pontos reais da Península e outra com o endereço virando coordenada. Se vier 501 sem chave, o segredo não entrou (nome exato importa); se vier erro do Google, é a restrição da chave.

8. Preencher o pino dos clientes que já existem

Em 12/09/2026, **4 dos 9 clientes tinham endereço e nenhum pino** (3 já tinham pino e 2 não tinham endereço nenhum) — sem pino, eles não entram no mapa nem no cálculo de trânsito. O script lista exatamente quem são, então esse número sai do banco, não de estimativa.

```
cd /opt/data/pack-and-paws/mobile
node scripts/geocode-clients.mjs # simulação: mostra o que seria gravado
node scripts/geocode-clients.mjs --gravar # grava no banco
```

Daqui para frente é automático: **cliente novo com endereço** e **cliente cujo endereço muda** ganham o pino sozinhos — o app pede ao servidor em segundo plano, sem segurar a tela do gestor, e sem pino "chutado" quando o servidor não responde.

9. Armadilhas (todas já vistas)

que custa dinheiro se vazar.

br.com.automadigital.app).

chave continua válida depois da transferência.

segredo do Supabase.

devolveria o centro da cidade e o pino mandaria o motorista ao lugar errado. Sem pino é melhor que pino errado — e a navegação por endereço continua funcionando.

1. **Não** use a chave do servidor no app. A do app vive no binário (pública); a do servidor é a
2. Defina o **limite diário de cota**. Sem isso, um bug em laço vira fatura.
3. Nomes de segredo têm que ser exatos: GOOGLE_ROUTES_KEY e/ou GOOGLE_GEOCODING_KEY.
4. Mapa **cinza** no celular = restrição de app da CHAVE 1 com bundle ID errado (o certo é
5. O bundle ID **não muda** quando o app passa para a conta Apple do cliente — a restrição da
6. Nunca commite chave no repositório: ela vive no app.json do build (via script) ou como
7. Endereço sem número de rua (address_line_1 vazio) **não** é geocodado de propósito: o Google